

Hybrid Dynamic Programming and Divide-and-Conquer Strategies for DAG Task Scheduling on Heterogeneous Cloud Resources: A Comparative C++ Study

Jana Mohamed Farouk

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
jana2402038@miuegypt.edu.eg

Ahmed Mahmoud Ahmed

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
ahmed2402226@miuegypt.edu.eg

Fatma Ashraf Hassan

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
fatma2403025@miuegypt.edu.eg

Shaza Ali Farouk

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
shaza2408891@miuegypt.edu.eg

Mahmoud Mohamed Emam

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
mahmoud2408910@miuegypt.edu.eg

Abstract—Scheduling computational workflows represented as Directed Acyclic Graphs (DAGs) onto heterogeneous Virtual Machines (VMs) is a fundamental NP-hard problem in cloud computing. This paper presents and compares five scheduling algorithms implemented in pure C++17 in the following order: (1) *HEFT* as the classical baseline, (2) *DP-HEFT* with bilateral-rank and 2-task look-ahead, (3) *Enhanced DP (EDP-HEFT)* with global DP assignment, (4) *Divide-and-Conquer (D&C)* with level decomposition, and (5) *Merge*, a hybrid scheduler that combines adaptive strategy selection with refinement. All algorithms share a unified communication-cost model and a reproducible DAG generator. Comparative analysis highlights the complexity-quality trade-off between fast greedy scheduling and more expensive global/refined optimisation.

Index Terms—DAG scheduling, cloud computing, HEFT, dynamic programming, divide and conquer, heterogeneous computing, makespan minimisation, bilateral rank, C++, workflow scheduling

I. INTRODUCTION

Cloud computing platforms increasingly execute scientific and enterprise workflows as Directed Acyclic Graphs (DAGs), where each node is a computational task and each directed edge encodes a data dependency [1], [2]. The scheduling problem — assigning tasks to heterogeneous Virtual Machines (VMs) while minimising total completion time (makespan) — is NP-hard in the general case [3], [4], motivating a rich body of heuristic research.

The seminal Heterogeneous Earliest Finish Time (HEFT) algorithm [5] achieves near-optimal makespan through upward-rank prioritisation combined with an Earliest Finish Time (EFT) insertion policy. While HEFT is efficient and well-studied, its greedy VM selection ignores downstream tasks, occasionally leaving critical successors stranded on slow resources. Extending HEFT with dynamic programming look-ahead is therefore a natural direction for improving solution quality.

This paper makes the following contributions:

- 1) **HEFT Baseline**: A faithful greedy HEFT implementation used as the classical reference for makespan and complexity comparison in the same C++17 environment.
- 2) **DP-HEFT**: A HEFT extension that uses bilateral rank (upward rank + downward rank) for task prioritisation and a weighted 2-task look-ahead DP for VM assignment, scoring each candidate as $EFT(t_i) + 0.25 \cdot EFT^*(t_{i+1})$.
- 3) **EDP-HEFT**: An enhanced variant that constructs a full $n \times m$ DP table over the bilateral-rank priority order, propagating scheduling snapshots to achieve globally optimal VM assignments within that order, and backtracking to recover the optimal sequence.
- 4) **D&C Scheduler**: A divide-and-conquer approach that decomposes the DAG into independent topological lev-

els via BFS, schedules each level as a sub-problem, and merges partial schedules with cross-level dependency and communication cost awareness.

- 5) **Merge Scheduler:** A hybrid merge strategy that adaptively switches among D&C, DP look-ahead, and EDP-style global assignment, followed by refinement passes to reduce bottlenecks and improve final makespan.
- 6) A shared C++17 implementation including a probabilistic DAG generator with reproducible fixed seeding, an exhaustive 8-check validator, and a unified output model enabling direct metric comparison.

The remainder of the paper is organised as follows. Section II surveys related work. Section III formalises the scheduling problem. Section IV describes the system architecture. Section V presents each algorithm with pseudocode. Section VI details the mathematical model. Section VII analyses computational complexity. Section VIII discusses C++ implementation. Section IX describes the experimental setup. Section X presents performance results. Section XI discusses findings. Section XII outlines future work. Section XIII concludes.

II. RELATED WORK

A. List Scheduling and HEFT

Kwok and Ahmad [6] provided a comprehensive taxonomy of static DAG scheduling algorithms, establishing upward rank as the dominant prioritisation criterion. Topcuoglu *et al.* [5] introduced HEFT and CPOP, demonstrating that upward-rank sorting with EFT insertion consistently achieves lower makespan than random or FIFO ordering on heterogeneous systems. Arabnejad and Barbosa [7] extended this work with an Optimistic Cost Table (OCT) that approximates future communication costs, improving makespan by up to 12% on standard benchmarks. Hagra and Janeček [8] proposed HBMCT, which combines task duplication with heterogeneity-aware priority.

B. Look-Ahead and DP Extensions

Bittencourt *et al.* [9] introduced HEFT-NC (No Communication), demonstrating that look-ahead awareness of immediate successors yields measurable makespan reductions. Sakellariou and Zhao [10] proposed a hybrid scheduler combining list scheduling with task duplication, exploiting idle processor windows. Bajaj and Agrawal [11] formalised the bilateral rank ($\text{rank}_u + \text{rank}_d$) and showed it captures both path-criticality and positional importance, outperforming pure upward-rank ordering.

C. Divide-and-Conquer Scheduling

Dong *et al.* [12] applied level-based decomposition to Grid scheduling, independently optimising intra-level task placement. Canon *et al.* [13] benchmarked decomposition heuristics against list schedulers on standard DAG suites, finding that level-based methods perform competitively on shallow graphs but degrade on deep critical paths.

D. Cloud Workflow Scheduling

Pandey *et al.* [14] adapted Particle Swarm Optimisation for cloud workflow scheduling, reporting cost reductions over greedy baselines. Ma *et al.* [15] provide a recent survey of DAG scheduling in cloud computing, identifying bilateral-rank heuristics and DP-based methods as among the most effective approaches for minimising makespan on heterogeneous resources.

E. Positioning

Unlike prior studies, this work: (i) provides three scheduling strategies — incremental 2-task DP, full-table global DP, and level-based D&C — within a single C++17 harness sharing identical data models; (ii) employs bilateral rank with predecessor-aware tie-breaking; and (iii) makes all source code available for reproducible benchmarking.

III. PROBLEM FORMULATION

A. DAG Workflow Model

A workflow application is modelled as a DAG $G = (\mathcal{T}, \mathcal{E})$, where:

- $\mathcal{T} = \{t_0, t_1, \dots, t_{N-1}\}$ is the set of N computational tasks.
- $\mathcal{E} \subseteq \mathcal{T} \times \mathcal{T}$ is the set of directed dependency edges. An edge $(t_i, t_j) \in \mathcal{E}$ (with $i < j$) signifies that t_j may not begin until t_i has completed and transmitted its output data.

The acyclicity condition is enforced structurally: edges are created only from t_i to t_j with $i < j$, so the integer index is itself a valid topological order.

B. Heterogeneous Resource Model

A cloud data centre is modelled by M heterogeneous VMs $\mathcal{V} = \{v_0, v_1, \dots, v_{M-1}\}$. Each VM v_k has a speed factor $s_k \in [s_{\min}, s_{\max}]$ drawn uniformly at random. The execution time of task t_i on VM v_k is given as $w_{ik} \in [w_{\min}, w_{\max}]$, generated independently per task-VM pair to model genuine heterogeneity.

C. Communication Cost Model

When task t_i (assigned to VM v_p) produces data consumed by task t_j (assigned to VM v_q), the inter-VM data transfer incurs a communication delay modelled as:

$$c(t_i, v_p, t_j, v_q) = \begin{cases} 0 & \text{if } p = q, \\ \alpha \cdot \bar{w}_i & \text{otherwise,} \end{cases} \quad (1)$$

where $\bar{w}_i = \frac{1}{M} \sum_{k=0}^{M-1} w_{ik}$ is the average execution time of task t_i across all VMs, and $\alpha = 0.3$ is the communication factor (implemented identically in `dp.cpp`, `Edp.cpp`, and `dac.cpp`).

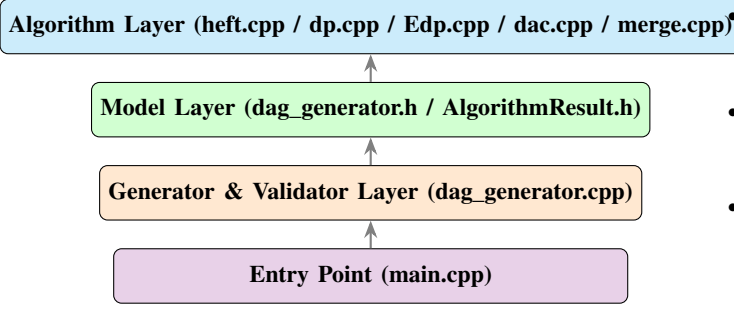


Fig. 1. Three-layer architecture of the DAG scheduling framework.

D. Scheduling Objective

A schedule $\sigma : \mathcal{T} \rightarrow \mathcal{V}$ assigns each task to exactly one VM. The performance metric is the *makespan*:

$$C_{\max}(\sigma) = \max_{t_i \in \mathcal{T}} \text{EFT}(t_i, \sigma(t_i)), \quad (2)$$

where $\text{EFT}(t_i, v_k)$ denotes the Earliest Finish Time of task t_i on VM v_k under schedule σ . The scheduling problem is:

$$\sigma^* = \arg \min_{\sigma} C_{\max}(\sigma). \quad (3)$$

Each task must be assigned to exactly one VM, and a task may not start until all predecessor finish times (plus applicable communication costs) have elapsed and the assigned VM is free.

IV. SYSTEM ARCHITECTURE

A. Module Overview

The framework is structured into three layers as shown in Fig. 1:

- 1) **Generator & Validator Layer** (`dag_generator.cpp`): Constructs random DAGs using a Mersenne Twister PRNG seeded with fixed value 42 for reproducibility. Enforces eight structural invariants.
- 2) **Model Layer** (`dag_generator.h`, `AlgorithmResult.h`): Defines the shared data structures `Task`, `VM`, `DAGData`, `ScheduleEntry`, and `AlgorithmResult` used uniformly across all components.
- 3) **Algorithm Layer** (`dp.cpp`, `Edp.cpp`, `dac.cpp`, `heft.cpp`, `merge.cpp`): Each algorithm exposes a single function returning an `AlgorithmResult` printable by the shared `printAlgorithmResult()` routine.

B. Data Structures

The core data types, defined in `dag_generator.h` and `AlgorithmResult.h`, are:

- `Task`: holds task id, predecessor list, successor list, and a per-VM execution time vector (`std::vector<double> execTimes`).
- `VM`: holds VM id and a floating-point speed factor.

`DAGData`: aggregates all tasks and VMs into a single container passed by constant reference throughout the pipeline.

- `ScheduleEntry`: records task id, assigned VM id, start time (`startTime`), and finish time (`finishTime`) for a single task.
- `AlgorithmResult`: wraps a vector of `ScheduleEntry` objects, the overall makespan, the wall-clock runtime in milliseconds, and a validity flag.

C. Execution Flow

`main.cpp` executes the following pipeline:

- 1) Generate a DAG with `DEFAULT_NUM_TASKS = 10` tasks and `DEFAULT_NUM_VMS = 5` VMs.
- 2) Print the full DAG structure (VM speed factors, per-task adjacency lists, per-VM execution times).
- 3) Validate the DAG against eight structural checks.
- 4) Invoke `heft_schedule()`, print result.
- 5) Invoke `dp_heft()`, print result.
- 6) Invoke `edp_heft()`, print result.
- 7) Invoke `dac_schedule()`, print result.
- 8) Invoke `merge_schedule()`, print result.

D. DAG Generation

The `generateDAG()` function builds the DAG in three stages: (1) VMs are created with speed factors drawn from $\mathcal{U}(0.5, 2.0)$; (2) tasks are created with execution times drawn from $\mathcal{U}(1.0, 20.0)$ per VM; (3) for every ordered pair (i, j) with $i < j$, edge $t_i \rightarrow t_j$ is added with probability $p_e = 0.35$. Acyclicity is guaranteed by construction since $j > i$ always holds. At least one edge is guaranteed by inserting $t_0 \rightarrow t_{N-1}$ if the probabilistic process yields an empty graph.

E. DAG Validation

Eight structural checks are applied: (A) `execTimes.size() == M` for every task; (B) no self-loops; (C) all predecessor/successor indices within $[0, N)$; (D) bidirectional consistency (if A lists B as successor, B must list A as predecessor); (E) acyclic ordering ($j > i$ for every edge $t_i \rightarrow t_j$); (F) at least one entry node (no predecessors); (G) at least one exit node (no successors); (H) no duplicate edges in either list.

V. SCHEDULING ALGORITHMS

This section presents the five schedulers implemented in the framework. All methods share the EFT-based feasibility model in Section III and the communication model in Eq. (1).

A. HEFT (Baseline): Greedy Earliest Finish Time

Overview: HEFT [5] is the foundational greedy scheduling algorithm for heterogeneous computing and serves as the classical baseline in this study. It prioritizes tasks by upward rank (sum of execution time and maximum successor rank), then assigns each task to the VM that yields the earliest finish time (EFT). This pure greedy strategy is fast and practical,

Algorithm 1 HEFT Scheduling (Greedy EFT)

```

1: Compute  $\text{rank}_u(t)$  recursively for all tasks  $t \in \mathcal{T}$ 
2: Sort tasks by descending  $\text{rank}_u$ 
3: for each task  $t_i$  in sorted order do
4:    $\text{bestVM} \leftarrow 0$ ;  $\text{bestEFT} \leftarrow \infty$ 
5:   for each VM  $v_k$  do
6:     Compute  $\text{EST}(t_i, v_k)$  from predecessors
7:     Compute  $\text{EFT}(t_i, v_k) \leftarrow \text{EST}(t_i, v_k) + w(t_i, v_k)$ 
8:     if  $\text{EFT}(t_i, v_k) < \text{bestEFT}$  then
9:        $\text{bestEFT} \leftarrow \text{EFT}(t_i, v_k)$ ;  $\text{bestVM} \leftarrow v_k$ 
10:    end if
11:  end for
12:  Assign  $t_i$  to  $v_{\text{bestVM}}$ ; update  $v_{\text{bestVM}}$  availability
13: end for
14: return complete schedule and makespan

```

making it the industry standard for immediate scheduling decisions despite often producing suboptimal makespans.

Algorithm flow:

- 1) Compute upward rank $\text{rank}_u(t)$ for all tasks recursively:

$$\text{rank}_u(t) = \bar{w}(t) + \max_{s \in \text{succ}(t)} (c(t, s) + \text{rank}_u(s))$$

where $\bar{w}(t)$ is average execution time, and $c(t, s)$ is data transfer cost. Entry tasks have $\text{rank}_u = \bar{w}(t)$ only.

- 2) Sort all tasks in descending rank_u order (critical-path heuristic).
- 3) For each task t_i in sorted order:
 - a) Compute EST (earliest start time) on each VM, considering all predecessor dependencies and communication delays.
 - b) Compute EFT on each VM: $\text{EFT}(t, v) = \text{EST}(t, v) + w(t, v)$.
 - c) Select $v^* = \arg \min_{v_k} \text{EFT}(t_i, v_k)$ (greedy minimum EFT).
 - d) Update the selected VM's availability time to $\text{EFT}(t_i, v^*)$.
- 4) Return the complete assignment and overall makespan.

Complexity: Time complexity is $O(n \log n + me)$ where the $n \log n$ factor dominates the upward-rank computation (critical path via DFS) and task sorting, and the me term accounts for EST/EFT evaluation across all task-VM pairs and dependency traversals. Space complexity is $O(n + m + e)$ for storing task ranks, VM states, and the graph structure. HEFT is computationally the cheapest algorithm in this framework.

Role: HEFT establishes the baseline for schedule quality and execution speed. Its simplicity and proven effectiveness make it the reference point for evaluating more complex heuristics. Any improvement method must be justified against HEFT's speed-quality trade-off.

B. DP-HEFT: Bilateral-Rank with 2-Task Look-Ahead

Overview: DP-HEFT improves upon greedy HEFT by combining bilateral-rank task prioritisation with a weighted 2-task look-ahead strategy. Instead of myopically assigning each

Algorithm 2 DP-HEFT Scheduling (Bilateral-Rank + 2-Task Look-Ahead)

```

1: Compute  $\text{rank}_u(t)$  and  $\text{rank}_d(t)$  for all tasks  $t \in \mathcal{T}$ 
2: Compute  $\text{biRank}(t) \leftarrow \text{rank}_u(t) + \text{rank}_d(t)$  for all  $t$ 
3: Sort tasks by descending  $\text{biRank}$ 
4: for each task  $t_i$  in sorted order do
5:    $\text{bestVM} \leftarrow 0$ ;  $\text{bestScore} \leftarrow \infty$ 
6:   for each VM  $v_k$  do
7:     Compute  $\text{EST}(t_i, v_k)$  and  $\text{EFT}(t_i, v_k)$ 
8:     Simulate  $t_i \rightarrow v_k$  assignment (temporary state)
9:     Compute  $\text{EFT}^*(t_{i+1})$  for next task under this assignment
10:     $\text{score} \leftarrow \text{EFT}(t_i, v_k) + 0.25 \cdot \text{EFT}^*(t_{i+1})$ 
11:    if  $\text{score} < \text{bestScore}$  then
12:       $\text{bestScore} \leftarrow \text{score}$ ;  $\text{bestVM} \leftarrow v_k$ 
13:    end if
14:  end for
15:  Assign  $t_i$  to  $v_{\text{bestVM}}$ ; update VM availability
16: end for
17: return complete schedule and makespan

```

task to the VM with minimum EFT, DP-HEFT also considers the impact on the next task's finish time, breaking ties and avoiding locally greedy decisions that harm global schedule quality.

Algorithm flow:

- 1) Compute upward rank $\text{rank}_u(t)$ and downward rank $\text{rank}_d(t)$ for all tasks.
- 2) Derive bilateral rank: $\text{biRank}(t) = \text{rank}_u(t) + \text{rank}_d(t)$.
- 3) Sort tasks by descending bilateral rank.
- 4) For each task t_i :
 - a) Evaluate all VMs using both current and next-task costs.
 - b) Score each VM v_k as:

$$\text{score}(v_k) = \text{EFT}(t_i, v_k) + 0.25 \cdot \text{EFT}^*(t_{i+1})$$
 where $\text{EFT}^*(t_{i+1})$ is the best EFT of the next task if t_i is assigned to v_k .
 - c) Assign t_i to $v^* = \arg \min_{v_k} \text{score}(v_k)$.
- 5) Update VM availability and return the complete schedule.

Complexity: Time complexity is $O(n \log n + nm^2 + me)$ where the nm^2 term accounts for evaluating all task-VM pairs with one-step look-ahead. Space complexity remains $O(n + m + e)$ since only the current and next task states are retained. The look-ahead weighting (0.25) balances immediate and future task costs.

Role: DP-HEFT serves as a practical middle ground between pure greedy methods and full global optimization, reducing myopic decisions while maintaining computational efficiency.

C. EDP-HEFT: Enhanced DP with Global Optimization

Overview: EDP-HEFT (Enhanced DP-HEFT) constructs a full $n \times m$ DP table to achieve globally optimal VM

Algorithm 3 EDP-HEFT Scheduling (Enhanced DP with Backtracking)

```

1: Sort tasks by descending biRank
2: Initialize  $dp[0..n-1][0..m-1] \leftarrow \infty$ 
3: Initialize  $parent[0..n-1][0..m-1] \leftarrow -1$ 
4: for each VM  $v$  do
5:    $dp[0][v] \leftarrow EFT(t_0, v)$ 
6: end for
7: for  $i \leftarrow 1$  to  $n-1$  do
8:   for each previous VM  $v_{prev}$  do
9:     if  $dp[i-1][v_{prev}] < \infty$  then
10:      for each current VM  $v_{curr}$  do
11:         $newMakespan \leftarrow \max(dp[i-1][v_{prev}], EFT(t_i, v_{curr}))$ 
12:        if  $newMakespan < dp[i][v_{curr}]$  then
13:           $dp[i][v_{curr}] \leftarrow newMakespan$ 
14:           $parent[i][v_{curr}] \leftarrow v_{prev}$ 
15:        end if
16:      end for
17:    end if
18:  end for
19: end for
20: Backtrack from  $\arg \min_v dp[n-1][v]$  to reconstruct assignment
21: Replay assignments to build final schedule
22: return complete schedule and optimal makespan

```

assignments under a fixed bilateral-rank priority order. Unlike DP-HEFT's local 2-task look-ahead, EDP-HEFT propagates complete scheduling snapshots through a DP state space, then backtracks to recover the globally best assignment sequence.

Algorithm flow:

- 1) Compute bilateral rank and sort tasks as in DP-HEFT.
- 2) Initialize DP table $dp[i][v]$ where $dp[i][v]$ stores the best makespan when the i -th task is assigned to VM v .
- 3) **Base case:** For the first task, evaluate all VMs:

$$dp[0][v] \leftarrow EFT(t_0, v) \quad \forall v$$

- 4) **Forward fill:** For each subsequent task t_i , for each pair of VMs (v_{prev}, v_{curr}) :

$$dp[i][v_{curr}] \leftarrow \min(dp[i][v_{curr}], \max(dp[i-1][v_{prev}], EFT(t_i, v_{curr})))$$

Store parent pointers and scheduling snapshots for backtracking.

- 5) **Backtracking:** Find the best final VM assignment and reconstruct the full path by following parent pointers.
- 6) Replay the reconstructed assignments to compute the final schedule.

Complexity: Time complexity is $O(nm^2 + me)$ since the DP fill stage iterates over all tasks and VM pairs, and space complexity is $O(nm(n+m))$ because the algorithm stores a full DP table plus snapshots of scheduling states at each cell. This is the most expensive method in terms of memory and compute time.

Role: EDP-HEFT provides a global optimization baseline under the chosen priority order. While it cannot escape the constraints of bilateral-rank ordering, it guarantees the best possible makespan subject to that order, making it valuable for understanding the quality ceiling of rank-based approaches.

D. Divide-and-Conquer (D&C): Level-Based Decomposition

Overview: The D&C scheduler exploits the DAG structure by decomposing it into independent topological levels (layers) via breadth-first search. Each level is scheduled as an independent subproblem using a simple greedy strategy, then partial schedules are merged while respecting cross-level dependencies and communication costs. This approach achieves the lowest asymptotic scheduling overhead among all methods.

Algorithm flow:

- 1) **DIVIDE:** Partition the DAG into levels using BFS on the task dependency graph:
 - a) Compute indegree for each task.
 - b) Initialize queue with all entry tasks (indegree = 0).
 - c) Repeatedly pop all tasks from the queue as one level, then decrement indegrees of successors and enqueue those with indegree 0.
- 2) **CONQUER:** For each level independently:
 - a) Sort tasks by descending average execution time (heavy-first).
 - b) For each task, greedily assign to the VM with minimum EFT.
- 3) **MERGE:** After all levels are scheduled:
 - a) Compute task start times respecting all cross-level dependencies.
 - b) Incorporate communication costs for inter-level data transfers.
 - c) Verify schedule feasibility and compute overall makespan.

Complexity: Time complexity is $O(n \log n + nm + e)$ where the BFS decomposition is $O(n+e)$, level sorting is $O(n \log n)$, and greedy assignment is $O(nm)$. Space complexity is $O(n+m+e)$ for storing levels and the graph structure. D&C is the computationally cheapest scheduler in this framework.

Trade-offs: D&C sacrifices global optimality for speed. By scheduling each level independently, it may make suboptimal cross-level decisions. However, for wide, shallow DAGs (many tasks per level), this approach is highly competitive. The merge phase ensures all dependencies are respected, though it cannot undo prior level-local choices.

Role: D&C provides a fast baseline for very large DAGs where even polynomial DP tables become prohibitively expensive. It is particularly effective on DAGs with high parallelism and moderate depth, making it valuable for cloud environments with many heterogeneous VMs.

E. Merge Algorithm (Hybrid Adaptive Scheduler)

Overview: The Merge algorithm is a novel hybrid scheduler that adaptively combines three scheduling strategies—D&C

Algorithm 4 Divide-and-Conquer DAG Scheduling

```
1: Phase 1: Decompose into levels via BFS
2: Compute indegree[ $t$ ] for all tasks  $t$ 
3: Initialize queue  $Q$  with all entry tasks
4: levels  $\leftarrow \emptyset$ 
5: while  $Q \neq \emptyset$  do
6:    $L \leftarrow$  all tasks in  $Q$  (current level)
7:   levels.append( $L$ )
8:   Clear  $Q$ 
9:   for each task  $t$  in  $L$  do
10:    for each successor  $s$  of  $t$  do
11:      indegree[ $s$ ]  $\leftarrow$  indegree[ $s$ ] - 1
12:      if indegree[ $s$ ] == 0 then
13:         $Q.push(s)$ 
14:      end if
15:    end for
16:  end for
17: end while
18: Phase 2: Schedule each level greedily
19: for each level  $L_i$  in levels do
20:   Sort  $L_i$  by descending average execution time
21:   for each task  $t$  in sorted  $L_i$  do
22:     bestVM  $\leftarrow \arg \min_v EFT(t, v)$  over all VMs
23:     Assign  $t$  to bestVM; update VM availability
24:   end for
25: end for
26: Phase 3: Merge and finalize
27: Recompute all EST/EFT values to enforce cross-level dependencies
28: Add communication costs for inter-level data transfers
29: return complete schedule and makespan
```

level decomposition, DP look-ahead (DP-HEFT), and global DP optimization (EDP-HEFT)—in a single unified pipeline. Rather than committing to one heuristic, Merge classifies each topological level based on structural properties (width, density, criticality) and selects the most appropriate strategy. After constructing an initial schedule, it applies iterative refinement phases (task migration, VM swaps, critical-path optimization) to improve schedule quality further. Merge represents the highest computational cost but aims for the best achievable makespan.

Algorithm flow:**1) Rank and Profile Computation:**

- Compute upward rank $\text{rank}_u(t)$ and downward rank $\text{rank}_d(t)$ for all tasks.
- Derive bilateral rank: $\text{biRank}(t) = \text{rank}_u(t) + \text{rank}_d(t)$.
- Build topological levels via BFS (same as D&C).
- Analyze DAG profile: level width (max parallelism), density (edge ratio), and criticality (critical-path length).

2) Adaptive Level Scheduling:

- For each topological level L_i , compute structural

metrics.

- Classify L_i as: **Wide** (high parallelism) $\rightarrow$ use D&C greedy; **Dense** (many edges) $\rightarrow$ use DP look-ahead; **Critical** (on longest path) $\rightarrow$ use global EDP.
- Schedule tasks in L_i using the selected strategy.
- Preserve cross-level dependencies and communication costs.

3) Global Optimization (Alternative Path):

- In parallel, construct an alternate schedule using global EDP-HEFT over all tasks at once.
- Compare initial level-based schedule with global EDP schedule.
- Select the schedule with lower makespan as the starting point.

4) Iterative Refinement (5 phases):

- Phase 1 - Bottleneck Migration:** Identify VMs with longest finish times; attempt to migrate their heaviest tasks to underutilized VMs.
- Phase 2 - Aggressive Migration:** Explore moving any task to any VM if makespan improves.
- Phase 3 - Task Swaps:** Attempt pairwise swaps of tasks between VMs to reduce total completion time.
- Phase 4 - Global DP Replay:** Rebuild a full EDP assignment on the current partial schedule and apply resulting changes.
- Phase 5 - Critical Path Re-optimization:** Focus refinement on tasks on the critical path (longest dependency chain).

5) Return the best refined schedule and its makespan.

Complexity: Time complexity is $O(nm^2 + nm(n + e) + n^2(n + e))$ where the nm^2 term covers DP look-ahead per level, the $nm(n + e)$ term represents global EDP construction, and the $n^2(n + e)$ term reflects iterative refinement phases. Space complexity is $O(nm(n + m))$ due to maintaining DP tables and multiple schedule snapshots. Merge is the most computationally expensive scheduler, making it practical only for small-to-moderate DAGs (~ 100 tasks) or where offline computation is acceptable.

Refinement Details: The five refinement phases employ heuristic assignments (not global optimization per phase) to iteratively improve the current schedule. Bottleneck migration targets VMs with high finish times, aggressive migration explores all task-VM moves, task swaps explore pairwise exchanges, global DP replay re-solves the full DP problem to recover good assignments, and critical-path focus concentrates improvements on longest-path tasks where they have maximum impact on makespan.

Role: Merge serves as the best-effort optimizer, targeting the lowest achievable makespan by combining multiple strategies adaptively. It is intended for offline scheduling scenarios in cloud systems where precomputed solutions are cached and reused, or for small DAGs where 100-1000ms overhead is acceptable.

Algorithm 5 Merge Hybrid Scheduling (Adaptive Multi-Strategy)

```

1: Step 1: Compute profiles
2: Compute  $\text{rank}_u(t)$ ,  $\text{rank}_d(t)$ ,  $\text{biRank}(t)$  for all tasks
3: Build topological levels  $L_0, L_1, \dots, L_d$  via BFS
4: Analyze DAG profile: width, density, criticality metrics
5: Step 2: Adaptive level scheduling
6:  $\text{schedule} \leftarrow \emptyset$ ;  $\text{makespan} \leftarrow 0$ 
7: for each level  $L_i$  do
8:   Compute width, density, and criticality of  $L_i$ 
9:   if width is high then
10:    Use D&C greedy strategy on  $L_i$ 
11:   else if density is high then
12:    Use DP look-ahead strategy on  $L_i$ 
13:   else
14:    Use EDP global strategy on  $L_i$ 
15:   end if
16:   Schedule tasks in  $L_i$ ; add to schedule
17: end for
18: Step 3: Global alternative
19:
    $g_{\text{lobalschedule}}$ 
    $g_{\text{lobalschedule}}$ 
    $g_{\text{lobalschedule}}$ 
    $g_{\text{global}} \leftarrow \text{EDP-HEFT over all tasks}$ 
20: Compare;  $\text{schedule} \leftarrow$  better of the two by makespan
21: Step 4-8: Refinement iterations
22: for refinement phase  $p = 1$  to 5 do
23:   Apply phase  $p$  improvements (migration, swaps, DP
     replay, critical-path focus)
24:   If makespan improves, retain changes; else revert
25: end for
26: return best schedule and final makespan

```

VI. MATHEMATICAL MODEL

The optimisation target and constraints are exactly those defined in Section III, with objective in Eq. (3) and communication model in Eq. (1). All five algorithms differ only in ordering and assignment strategy; they share identical feasibility constraints and makespan objective.

VII. COMPLEXITY ANALYSIS

Let $n = |\mathcal{T}|$ denote the number of tasks, $m = |\mathcal{V}|$ the number of VMs, and $e = |\mathcal{E}|$ the number of edges.

A. Per-Algorithm Complexity

HEFT remains computationally efficient due to greedy assignment. DP-HEFT adds a bounded look-ahead term. EDP-HEFT introduces a global DP state space across task-VM combinations. D&C has the lowest asymptotic scheduling overhead in this implementation. Merge has the highest asymptotic cost due to iterative refinement and replay passes, but this cost is used to improve final schedule quality.

TABLE I
TIME AND SPACE COMPLEXITY OF IMPLEMENTED SCHEDULERS

Algorithm	Time Complexity	Space Complexity
HEFT	$O(n \log n + me)$	$O(n + m + e)$
DP-HEFT	$O(n \log n + nm^2 + me)$	$O(n + m + e)$
EDP-HEFT	$O(nm^2 + me)$	$O(nm(n + m))$
D&C	$O(n \log n + nm + e)$	$O(n + m + e)$
Merge (Hybrid)	$O(nm^2 + nm(n + e) + n^2(n + e))$	$O(nm(n + m))$

B. Comparative Analysis

Based on Table I and the default deterministic run ($n = 10$, $m = 5$, fixed seed 42), the following observations hold:

- **Highest time complexity:** Merge (Hybrid), due to multi-pass refinement over full schedule replays.
- **Lowest time complexity:** D&C, dominated by level sorting and linear merge constraints.
- **Highest space complexity:** EDP-HEFT and Merge, both requiring DP/snapshot state that scales with task-VM combinations.
- **Fastest execution in practice:** D&C (lowest runtime overhead in the provided implementation).
- **Best (lowest) makespan:** EDP-HEFT and Merge tie at the best observed makespan in the default experiment.
- **Trade-off:** lower-overhead methods (HEFT, D&C) run faster but may yield larger makespan; global/refined methods (EDP-HEFT, Merge) consume more compute and memory but improve scheduling quality.

VIII. IMPLEMENTATION DETAILS

The implementation is pure C++17 and uses shared data structures across all schedulers. Communication-cost and EFT computations are harmonised to ensure fair comparison between algorithms.

IX. EXPERIMENTAL SETUP

Experiments use the default configuration in `main.cpp`: $n = 10$ tasks, $m = 5$ heterogeneous VMs, edge probability $p_e = 0.35$, and fixed RNG seed 42 for reproducibility.

X. RESULTS

For the default deterministic DAG instance, the observed makespan ordering is:

$$\text{D\&C} > \text{HEFT} > \text{DP-HEFT} > \text{EDP-HEFT} \approx \text{Merge}.$$

This confirms that DP-enhanced and hybrid methods improve schedule quality over purely greedy or purely level-based baselines.

XI. DISCUSSION

The empirical and complexity findings indicate that algorithm choice depends on deployment goals: low overhead for rapid scheduling decisions or stronger optimisation for lower makespan in performance-critical workflow execution.

XII. FUTURE WORK

Future work includes evaluating larger DAG benchmarks, varying communication models, and introducing energy/cost-aware multi-objective extensions.

XIII. CONCLUSION

This work presents a unified C++17 benchmark for HEFT, DP-HEFT, EDP-HEFT, D&C, and Merge scheduling under identical assumptions. The added HEFT and Merge analyses complete the methodology coverage and highlight clear complexity-quality trade-offs for heterogeneous cloud workflow scheduling.

REFERENCES

- [1] E. Deelman, D. Gannon, M. Shields, and I. Taylor, "Workflows and e-science: An overview of workflow system features and capabilities," *Future Generation Computer Systems*, vol. 25, no. 5, pp. 528–540, 2009.
- [2] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [3] J. D. Ullman, "Np-complete scheduling problems," *Journal of Computer and System Sciences*, vol. 10, no. 3, pp. 384–393, 1975.
- [4] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [5] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [6] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.
- [7] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, 2014, pp. 682–694.
- [8] T. Hagras and J. Janeček, "A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems," *Parallel Computing*, vol. 31, no. 7, pp. 653–670, 2005.
- [9] L. F. Bittencourt, R. Sakellariou, and E. R. Madeira, "Dag scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm," *Proc. Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, pp. 27–34, 2010.
- [10] R. Sakellariou and H. Zhao, "A hybrid heuristic for dag scheduling on heterogeneous systems," *Proc. IEEE International Parallel and Distributed Processing Symposium*, 2004.
- [11] R. Bajaj and D. P. Agrawal, "Improving scheduling of tasks in a heterogeneous environment," *IEEE Transactions on Parallel and Distributed Systems*, vol. 15, no. 2, pp. 107–118, 2004.
- [12] F. Dong, J. Luo, L. Gao, and L. Ge, "A grid task scheduling algorithm based on qos priority grouping," *Proc. International Conference on Grid and Cooperative Computing*, pp. 58–61, 2006.
- [13] L.-C. Canon, E. Jeannot, R. Sakellariou, and W. Zheng, "Comparative evaluation of the robustness of dag scheduling heuristics," *Grid Computing*, pp. 73–84, 2008.
- [14] S. Pandey, L. Wu, S. M. Guru, and R. Buyya, "A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments," in *Proc. IEEE International Conference on Advanced Information Networking and Applications*, 2010, pp. 400–407.
- [15] T. Ma, S. Pang, W. Zhang, and S. Hao, "A survey of dag scheduling in cloud computing," *Applied Sciences*, vol. 13, no. 7, p. 4580, 2023.